

# GUIDA FORMATORE - CAPITOLO 3

---

## PROMPT ENGINEERING E CONTEXT MANAGEMENT

Corso: Claude Desktop e Claude for Code

Versione: 1.0 | Data: Gennaio 2026

## Sommario

|                                                          |    |
|----------------------------------------------------------|----|
| PROMPT ENGINEERING E CONTEXT MANAGEMENT .....            | 1  |
| 1. INTRODUZIONE AL CAPITOLO .....                        | 4  |
| 1.1 Panoramica e Obiettivi .....                         | 4  |
| Contesto del Capitolo .....                              | 4  |
| Obiettivi di Apprendimento .....                         | 4  |
| Perché Questo Capitolo è Fondamentale .....              | 5  |
| 🇮🇹 Note per il Formatore - Gestione della Sessione ..... | 6  |
| 2. FONDAMENTI TEORICI DEL PROMPT ENGINEERING .....       | 7  |
| 2.1 Cos'è il Prompt Engineering .....                    | 7  |
| Definizione Operativa .....                              | 7  |
| Analogia Accessibile .....                               | 8  |
| 2.3 Principi Cardine del Prompt Engineering .....        | 9  |
| I 5 Pilastri Fondamentali .....                          | 9  |
| PRINCIPIO 1: CHIAREZZA (Clarity) .....                   | 9  |
| PRINCIPIO 2: SPECIFICITÀ (Specificity) .....             | 11 |
| PRINCIPIO 3: CONTESTO (Context) .....                    | 12 |
| PRINCIPIO 4: FORMATO (Format) .....                      | 12 |
| PRINCIPIO 5: ESEMPI (Examples) .....                     | 13 |
| Applicare i 5 Principi: Framework Rapido .....           | 14 |
| 10. ESERCITAZIONI PRATICHE .....                         | 15 |
| 10.1 Esercizio 1 - Principiante .....                    | 15 |
| OBIETTIVO APPRENDIMENTO .....                            | 15 |
| DURATA .....                                             | 15 |
| MATERIALE NECESSARIO .....                               | 15 |
| TASK .....                                               | 15 |
| CRITERI DI SUCCESSO .....                                | 15 |
| HINT .....                                               | 15 |
| 10.2 Esercizio 2 - Intermedio .....                      | 17 |
| OBIETTIVO APPRENDIMENTO .....                            | 17 |
| DURATA .....                                             | 17 |
| TASK .....                                               | 17 |
| REQUISITI .....                                          | 17 |

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 12. BEST PRACTICES E CHECKLIST .....                                     | 18 |
| Checklist Pre-Prompt (Prima di Inviare) .....                            | 18 |
| Best Practices Summary .....                                             | 19 |
| DO: .....                                                                | 19 |
| DON'T: .....                                                             | 19 |
| 13. TROUBLESHOOTING .....                                                | 20 |
| Problemi Comuni e Soluzioni .....                                        | 20 |
| PROBLEMA 1: Output Generico/Vago .....                                   | 20 |
| PROBLEMA 2: Output Troppo Lungo .....                                    | 21 |
| PROBLEMA 3: Output nel Formato Sbagliato .....                           | 21 |
| 14. INTEGRAZIONE CON SPEC-DRIVEN DEVELOPMENT .....                       | 22 |
| Come Prompt Engineering è Fondamento dello Spec-Driven Development ..... | 22 |
| Mapping Principi Prompt Engineering → Spec Writing .....                 | 22 |
| Preview Capitolo 4 .....                                                 | 23 |
| 16. MATERIALI DI SUPPORTO .....                                          | 24 |
| 16.1 Template Prompt Framework .....                                     | 24 |
| 16.2 Quick Reference Card - I 5 Principi .....                           | 25 |
| 16.3 Glossario Termini Chiave .....                                      | 25 |
| CONCLUSIONE E PROSSIMI PASSI .....                                       | 26 |

# 1. INTRODUZIONE AL CAPITOLO

## 1.1 Panoramica e Obiettivi

### **Messaggio Chiave per il Formatore**

"Il prompt engineering è la differenza tra usare Claude come un semplice chatbot e sfruttarlo come un potente strumento di produttività professionale. Questo capitolo trasforma i partecipanti da utenti casuali a prompt engineers competenti."

### **Contesto del Capitolo**

Il Capitolo 3 rappresenta un punto di svolta nel percorso formativo. Dopo aver imparato a navigare l'interfaccia e gestire progetti (Capitoli 1-2), i partecipanti acquisiscono ora le competenze fondamentali che determinano la qualità e l'efficacia di ogni interazione con Claude.

Il prompt engineering non è una "tecnica avanzata" riservata a esperti - è la competenza basilare che ogni utente di Claude deve padroneggiare per ottenere risultati professionali. Senza queste competenze, anche le funzionalità più avanzate di Claude producono risultati mediocri.

### **Obiettivi di Apprendimento**

Al termine di questo capitolo, i partecipanti saranno in grado di:

- **COMPETENZE COGNITIVE (Sapere)**
  - Comprendere i meccanismi con cui Claude interpreta i prompt
  - Conoscere i principi fondamentali del prompt engineering efficace
  - Identificare le caratteristiche di prompt ben costruiti vs problematici
  - Comprendere la relazione tra prompt design e context management
- **COMPETENZE OPERATIVE (Saper Fare)**
  - Costruire prompt chiari, specifici e efficaci per diversi tipi di task
  - Iterare sistematicamente sui prompt per migliorare i risultati
  - Diagnosticare e correggere prompt problematici
  - Applicare framework riutilizzabili per prompt professionali
  - Adattare i prompt al contesto e alle vincoli della context window
- **COMPETENZE STRATEGICHE (Saper Decidere)**
  - Valutare quando un prompt è "pronto" vs quando necessita raffinamento
  - Decidere il livello di dettaglio appropriato per diversi task
  - Bilanciare specificità e flessibilità nei prompt
  - Riconoscere quando un problema è nel prompt vs quando è nel task stesso

## Perché Questo Capitolo è Fondamentale

### 1. 1. Competenza Trasversale

Il prompt engineering non è una funzionalità isolata ma la skill fondamentale che permea OGNI utilizzo di Claude:

- Scrivere codice con Claude for Code → Richiede prompt engineering
- Analizzare documenti → Richiede prompt engineering
- Automatizzare workflow → Richiede prompt engineering
- Implementare spec-driven development → Richiede prompt engineering avanzato

### 2. 2. Moltiplicatore di Efficacia

La differenza tra un prompt mediocre e uno eccellente può essere:

- 10x tempo: Da 10 iterazioni a 1 per ottenere il risultato desiderato
- 10x qualità: Da output generico a risultato perfettamente su misura
- 10x consistenza: Da risultati variabili a output predicibili

### 3. 3. Abilitatore di Funzionalità Avanzate

Tutte le funzionalità avanzate che verranno esplorate nei capitoli successivi dipendono da solide competenze di prompt engineering:

- Spec-driven development (Cap 4) richiede capacità di scrivere specifiche chiare
- Workflow patterns (Cap 5) si basano su prompt strutturati
- MCP servers (Cap 6) necessitano prompt precisi per orchestrare servizi esterni

## Note per il Formatore - Gestione della Sessione

### GESTIONE AUDIENCE ETEROGENEA:

Questo è probabilmente il capitolo dove le differenze di skill level si manifestano più chiaramente.

- Partecipanti Principianti:
  - Potrebbero sentirsi sopraffatti dalla quantità di "regole"
  - Strategia: Enfatizza che i 5 principi cardine sono tutto ciò che serve all'inizio
  - Rassicura: "Non dovete memorizzare tutto, svilupperete intuizione con la pratica"
- Partecipanti Intermedi:
  - Potrebbero credere di saper già fare prompt engineering
  - Strategia: Mostra esempi dove piccoli cambiamenti producono risultati drammaticamente migliori
  - Challenge: "Vediamo se riuscite a migliorare questo prompt che sembra già buono"
- Partecipanti che Hanno già Usato Claude:
  - Potrebbero avere cattive abitudini consolidate
  - Strategia: Valida ciò che fanno di buono, poi mostra alternative più efficaci
  - Frame: "Quello che fate funziona, ecco come farlo funzionare meglio"

## 2. FONDAMENTI TEORICI DEL PROMPT ENGINEERING

### 2.1 Cos'è il Prompt Engineering

#### Definizione Operativa

**Prompt Engineering** è l'arte e la scienza di **comunicare efficacemente con modelli di linguaggio AI** per ottenere risultati che soddisfano precisamente le tue esigenze.

Non è:

- ❌ "Chiedere gentilmente"
- ❌ Usare parole magiche o formule segrete
- ❌ Una skill solo per tecnici o programmatori
- ❌ Necessariamente complesso o lungo

È:

- ✅ Strutturare richieste in modo chiaro e non ambiguo
- ✅ Fornire contesto rilevante e sufficiente
- ✅ Specificare precisamente cosa vuoi ottenere
- ✅ Iterare sistematicamente per migliorare risultati
- ✅ Una competenza pratica che si sviluppa con l'esperienza

### Analogia Accessibile

💡 *Pensa al prompt engineering come dare istruzioni a un assistente molto competente ma che non ti conosce:*

### Assistente Umano vs Claude:

Quando chiedi a un collega di lunga data: "Fammi il solito report"

- ☒ Capisce cosa intendi per "solito"
- ☒ Conosce il formato che preferisci
- ☒ Sa quali dati includere
- ☒ Ricorda le tue preferenze stilistiche

Quando chiedi a Claude: "Fammi un report"

- ☒ Non sa quale report
- ☒ Non conosce il formato desiderato
- ☒ Non sa quali dati hai a disposizione
- ☒ Non conosce le tue preferenze

**La soluzione non è dire "Claude è stupido" - è fornire le informazioni che servono!**

Un prompt ben progettato trasforma "Fammi un report" in:

*"Crea un report di analisi vendite Q4 in formato tabellare con grafici a barre per ciascuna regione. Usa i dati dell'allegato sales\_Q4.xlsx. Includi: totali per regione, trend rispetto Q3, top 5 prodotti, e breve executive summary (3-4 frasi). Formato output: documento Word con stile professionale."*

## 2.3 Principi Cardine del Prompt Engineering

### I 5 Pilastri Fondamentali

Ogni prompt efficace si basa su questi 5 principi. Non servono tutti sempre, ma considerarli sistematicamente trasforma prompt mediocri in eccellenti.

#### PRINCIPIO 1: CHIAREZZA (Clarity)

**Definizione:** Essere espliciti e non ambigui su cosa vuoi.

##### Perché Importa:

Claude non legge nella mente. Se puoi interpretare una richiesta in multipli modi, Claude probabilmente sceglierà l'interpretazione sbagliata (Murphy's Law dell'AI).

##### Come Applicarlo:

✅ FARE:

- Usa verbi specifici: "Elenca", "Riassumi in 3 punti", "Crea tabella comparativa"
- Specifica l'obiettivo: "Per presentazione a management" vs "Per documentazione tecnica"
- Elimina ambiguità: "Analizza" → Troppo vago. "Calcola statistiche descrittive" → Chiaro.

❌ NON FARE:

- Verbi vaghi: "Controlla", "Guarda", "Fai qualcosa con"
- Assumere contesto: "Sai cosa intendo"
- Pronomi ambigui: "Analizza questo e quello" (quale "questo"?)

##### ESEMPIO - Prima e Dopo:

❌ **Prompt Vago:**

"Guarda questo documento e dimmi cosa ne pensi"

- Problemi:
  - "Guarda" → Cosa fare esattamente?
  - "dimmi cosa ne pensi" → Opinioni? Analisi? Riassunto? Critique?
  - Nessuna indicazione su formato risposta

✅ **Prompt Chiaro:**

"Analizza il documento allegato contract\_draft.docx e identifica:

1. Clausole potenzialmente problematiche per la nostra azienda
2. Termini che necessitano negoziazione
3. Sezioni mancanti secondo standard contrattuali

Formato risposta: Elenco numerato con citazioni dirette dal documento."

**Differenza:** Zero ambiguità su task e formato atteso.

## PRINCIPIO 2: SPECIFICITÀ (Specificity)

**Definizione:** Fornire dettagli sufficienti per guidare Claude verso il risultato esatto che vuoi.

### Perché Importa:

Specificità elimina "indovinare". Più dettagli fornisci (rilevanti!), più Claude può targetizzare precisamente le tue esigenze.

### Come Applicarlo:

#### ✅ FARE:

- Specifica formato output: "Tabella markdown", "Documento Word", "Email formale"
- Indica lunghezza: "3-5 frasi", "Circa 500 parole", "Executive summary 1 pagina"
- Definisci stile: "Tono professionale", "Linguaggio semplice", "Stile giornalistico"
- Nomina vincoli: "Non superare 1000 caratteri", "Evita gergo tecnico", "Usa solo dati dal file allegato"

#### ❌ NON FARE:

- Richieste generiche: "Scrivi un report" (che tipo? quanto lungo? per chi?)
- Assumere "formato ovvio": Non c'è formato ovvio, specificalo sempre
- Over-specificity su dettagli irrilevanti: Specificare font e margini esatti raramente serve

### PRINCIPIO 3: CONTESTO (Context)

**Definizione:** Fornire informazioni di background rilevanti che Claude ha bisogno per dare risposta appropriata.

#### **Perché Importa:**

Claude non conosce la tua situazione specifica. Contesto trasforma risposta generica in risposta personalizzata e rilevante.

#### *Tipi di Contesto:*

4. **Contesto Operativo:** Chi sei, cosa fai, per chi lavori
5. **Contesto del Task:** Perché stai chiedendo questo, obiettivo finale
6. **Contesto Tecnico:** Vincoli tecnologici, requisiti specifici
7. **Contesto Audience:** Chi leggerà/userà l'output

### PRINCIPIO 4: FORMATO (Format)

**Definizione:** Specificare esattamente come vuoi strutturato l'output.

#### **Perché Importa:**

Formato corretto significa output immediatamente usabile. Formato sbagliato significa dover riformattare manualmente → Tempo sprecato.

#### *Dimensioni di Formato:*

8. **Struttura:** Liste, tabelle, paragrafi, sezioni
9. **Medium:** Testo, Markdown, Word, Excel, codice
10. **Stile:** Formale, casual, tecnico, bullet points
11. **Layout:** Colonne, headers, indentazione

## PRINCIPIO 5: ESEMPI (Examples)

**Definizione:** Mostrare a Claude esempi di cosa vuoi, specialmente per stile/formato complessi.

### **Perché Importa:**

"Mostrare" è spesso più efficace di "dire". Esempi eliminano ambiguità residue e guidano stile.

### *Quando Usare Esempi:*

CRITICAL quando:

- Stile di scrittura specifico e non-standard
- Formato output complesso
- Pattern da replicare
- Tono particolare

NICE TO HAVE quando:

- Task standard (riassunto, analisi, etc.)
- Formato semplice (lista, paragrafo)

### Applicare i 5 Principi: Framework Rapido

CHECKLIST MENTALE prima di inviare prompt:

- 1. ☒ **CHIAREZZA:** È ovvio cosa sto chiedendo? Verbo specifico?
- 2. ☒ **SPECIFICITÀ:** Ho fornito dettagli sufficienti (formato, lunghezza, stile)?
- 3. ☒ **CONTESTO:** Claude ha background necessario per dare risposta appropriata?
- 4. ☒ **FORMATO:** Ho specificato esattamente come voglio strutturato l'output?
- 5. ☒ **ESEMPI:** (Se necessario) Ho mostrato esempio di cosa voglio?

### NON servono tutti e 5 sempre!

- Task semplice = 2-3 principi sufficienti
- Task complesso = Applica tutti e 5 sistematicamente

## 10. ESERCITAZIONI PRATICHE

### 10.1 Esercizio 1 - Principiante

#### OBIETTIVO APPRENDIMENTO

Applicare i 5 principi fondamentali per trasformare prompt vaghi in prompt efficaci

#### DURATA

30 minuti (20 min lavoro individuale + 10 min debriefing)

#### MATERIALE NECESSARIO

- Claude Desktop aperto
- File: customer\_feedback\_raw.txt (fornito)

#### TASK

Avete ricevuto feedback da 50 clienti in formato testuale non strutturato. Il vostro manager vuole un report che sintetizzi i feedback per board meeting di domani.

#### PROMPT INIZIALE (VAGO):

"Analizza questi feedback clienti e fammi un report"

#### VOSTRO COMPITO:

Trasformate questo prompt applicando i 5 principi:

12. 1. CHIAREZZA: Specificate verbo d'azione preciso, oggetto definito, risultato atteso

13. 2. SPECIFICITÀ: Definite formato, lunghezza, struttura

14. 3. CONTESTO: Aggiungete chi siete, per chi è il report, obiettivo

15. 4. FORMATO: Struttura desiderata output

16. 5. ESEMPI: (Opzionale per questo esercizio)

#### CRITERI DI SUCCESSO



Prompt risultante:

- Specifica chiaramente cosa analizzare
- Definisce formato output (sezioni, tabelle, etc)
- Include contesto rilevante (audience, obiettivo)
- Quando testato con Claude, produce report immediatamente usabile

#### HINT

Se bloccati, chiedetevi:

- "Cosa significa 'analizza' esattamente in questo contesto?"

- "Come vorrei strutturato il report? Sezioni? Tabelle? Liste?"
- "Chi leggerà questo report? Cosa gli serve sapere?"

## 10.2 Esercizio 2 - Intermedio

### OBIETTIVO APPRENDIMENTO

Costruire prompt complesso che orchestra multipli principi + gestione iterativa

### DURATA

40 minuti (30 min lavoro individuale + 10 min debriefing)

### TASK

Siete responsabili pricing strategy. Dovete analizzare performance Q4 e proporre aggiustamenti pricing per Q1 basati su: (1) Vostra performance Q4, (2) Competitor pricing, (3) Market trends.

Create prompt che:

17. 1. Analizza i due dataset
18. 2. Identifica insights chiave
19. 3. Propone raccomandazioni pricing
20. 4. Giustifica ogni raccomandazione con dati

### REQUISITI

Must Have:

- Context aziendale realistico
- Specificità su output format
- Chiare sezioni: Analysis → Insights → Recommendations
- Gestione di multipli file

Nice to Have:

- Esempi di insight style
- Template per raccomandazioni

## 12. BEST PRACTICES E CHECKLIST

### Checklist Pre-Prompt (Prima di Inviare)

#### CLARITY:

- ☐ Verbo d'azione è specifico e non ambiguo?
- ☐ Oggetto/target del prompt è chiaramente identificato?
- ☐ Risultato atteso è esplicitato?

#### SPECIFICITY:

- ☐ Lunghezza desiderata specificata?
- ☐ Formato/struttura definita?
- ☐ Stile/tono indicato?

#### CONTEXT:

- ☐ Background essenziale fornito?
- ☐ Audience identificata?
- ☐ Vincoli/requisiti esplicitati?

#### FORMAT:

- ☐ Struttura output chiara?
- ☐ Template fornito se necessario?
- ☐ Markup/formato tecnico specificato?

#### EXAMPLES:

- ☐ Esempi necessari? Se sì, inclusi?
- ☐ Esempi diversificati e annotati?
- ☐ Pattern evidenziato?

## Best Practices Summary

### DO:

-  Start specifico, aggiungi contesto upfront
-  Struttura prompt in sezioni chiare
-  Usa template per task ripetitivi
-  Itera sistematicamente
-  Documenta prompt che funzionano bene (riutilizzabili)

### DON'T:

-  Assumere Claude "capisce dal contesto"
-  Over-specify dettagli irrilevanti
-  Scrivere prompt come flusso di coscienza
-  Rinunciare dopo primo tentativo se output non perfetto
-  Dimenticare che prompt è iterativo

## 13. TROUBLESHOOTING

### Problemi Comuni e Soluzioni

#### PROBLEMA 1: Output Generico/Vago

##### SINTOMI:

- Risposta è corretta ma non utile
- Troppo high-level
- Mancano dettagli specifici

##### DIAGNOSI:

- Prompt probabilmente manca specificità o contesto

##### SOLUZIONE:

21. 1. Aggiungi contesto specifico situazione
22. 2. Specifica livello dettaglio desiderato
23. 3. Fornisci esempi di output desiderato
24. 4. Usa frasi come "Sii specifico", "Fornisci esempi concreti"

#### **ESEMPIO FIX:**

**Prima:** "Come posso migliorare le vendite?"

**Dopo:** "Analizza nostre vendite Q4 (data in allegato) e proponi 3 azioni specifiche per aumentare conversion rate entro 15%. Per ogni azione: descrivi implementazione, stima impatto, identifica risorse necessarie."

## PROBLEMA 2: Output Troppo Lungo

### SINTOMI:

- Claude scrive paragrafi quando servono bullet points
- Output supera lunghezza desiderata
- Informazioni ridondanti

### SOLUZIONE:

25. 1. Specifica lunghezza esplicitamente: "Max 200 parole", "5 bullet points"
26. 2. Usa frasi come "Concisamente", "Riassumi in"
27. 3. Definisci formato compatto: "Tabella", "Lista puntata"

## PROBLEMA 3: Output nel Formato Sbagliato

### SINTOMI:

- Paragrafi quando serviva tabella
- Markdown quando serviva Word
- Struttura diversa da desiderata

### SOLUZIONE:

28. 1. Fornisci template esatto da seguire
29. 2. Specifica formato markup: "Tabella markdown", "Documento Word"
30. 3. Mostra esempio di formato desiderato

## 14. INTEGRAZIONE CON SPEC-DRIVEN DEVELOPMENT

### Come Prompt Engineering è Fondamento dello Spec-Driven Development

#### KEY CONNECTION:

Una specifica funzionale efficace è essenzialmente un mega-prompt ultra-strutturato che applica tutti i principi di prompt engineering al massimo livello.

#### Mapping Principi Prompt Engineering → Spec Writing

- CHIAREZZA → Spec Requirements
  - Ogni requirement deve essere chiaro e non ambiguo
  - Acceptance criteria = Risultato atteso esplicito
  - Use cases = Verbi d'azione specifici
- SPECIFICITÀ → Detailed Specifications
  - Level of detail in spec = Specificità in prompt
  - Technical constraints = Vincoli espliciti
  - Success criteria = Formato e qualità desiderati
- CONTESTO → Background Section
  - Problem statement = Context del problema
  - User personas = Context audience
  - Technical environment = Context tecnico
- FORMATO → Spec Structure
  - Template spec = Template prompt
  - Sezioni standardizzate = Struttura consistente
  - Formatting conventions = Output format rules
- ESEMPI → Use Cases & Examples
  - Use case scenarios = Input-output examples
  - Edge case handling = Boundary examples
  - Mockups/wireframes = Visual examples

## Preview Capitolo 4

Nel prossimo capitolo vedrete come:

- Trasformare questi principi in metodologia strutturata
- Scrivere specifiche funzionali che producono implementazioni eccellenti
- Gestire progetti complessi con controllo granulare
- Iterare su specs come iterate su prompts

**TEASER:** *"Se oggi avete imparato a controllare precisamente cosa Claude fa in singolo prompt, nel Capitolo 4 imparerete a orchestrare Claude per progetti completi multi-step dove ogni step è controllato precisamente."*

## 16. MATERIALI DI SUPPORTO

### 16.1 Template Prompt Framework

Template Base per Prompt Strutturato

[TUO TASK/RICHIESTA PRINCIPALE]

**\*\*CONTEXT:\*\***

[Informazioni di background rilevanti]

- Chi sei/Ruolo
- Situazione/Problema
- Obiettivo finale

**\*\*REQUIREMENTS:\*\***

[Cosa deve contenere l'output]

1. [Requirement 1]
2. [Requirement 2]
3. [Requirement 3]

**\*\*FORMAT:\*\***

[Come strutturare output]

- Struttura: [Sezioni, tabelle, liste]
- Lunghezza: [Parole, frasi, pagine]
- Stile: [Formale, tecnico, casual]

**\*\*CONSTRAINTS:\*\***

[Limitazioni e vincoli]

- [Vincolo 1]
- [Vincolo 2]

**\*\*EXAMPLES:\*\*** (se necessario)

[Mostra 1-3 esempi di output desiderato]

## 16.2 Quick Reference Card - I 5 Principi

| PRINCIPIO      | DEFINIZIONE                   | DOMANDA CHIAVE                            |
|----------------|-------------------------------|-------------------------------------------|
| 1. CHIAREZZA   | Essere espliciti su cosa vuoi | È ovvio cosa sto chiedendo?               |
| 2. SPECIFICITÀ | Fornire dettagli sufficienti  | Ho specificato formato, lunghezza, stile? |
| 3. CONTESTO    | Background rilevante          | Claude ha info necessarie per rispondere? |
| 4. FORMATO     | Struttura output              | Come voglio organizzato l'output?         |
| 5. ESEMPI      | Mostrare cosa vuoi            | Serve mostrare esempio per chiarezza?     |

## 16.3 Glossario Termini Chiave

**Artifact:** Contenuto sostanziale creato da Claude (documenti, codice, visualizzazioni) che può essere scaricato e riutilizzato.

**Chiarezza:** Principio che richiede di essere espliciti e non ambigui nelle richieste a Claude.

**Context Window:** Lo "spazio di lavoro mentale" di Claude; la quantità totale di informazioni che può considerare simultaneamente.

**Few-Shot Prompting:** Tecnica che fornisce 2-5 esempi per guidare Claude verso output desiderato.

**Iterazione:** Processo di raffinamento progressivo del prompt attraverso cicli di test e miglioramento.

**Prompt:** Istruzione o richiesta fornita a Claude per ottenere un output specifico.

**Prompt Engineering:** L'arte e la scienza di comunicare efficacemente con modelli AI come Claude.

**Specificità:** Principio che richiede di fornire dettagli sufficienti (formato, lunghezza, stile) per guidare Claude.

**Template Prompt:** Struttura riutilizzabile di prompt che può essere adattata per task simili.

**Zero-Shot Prompting:** Tecnica che non fornisce esempi, solo descrizione del task.

## CONCLUSIONE E PROSSIMI PASSI

Questo Capitolo 3 ha fornito le competenze fondamentali di prompt engineering che trasformano l'utilizzo di Claude da casuale a professionale. I 5 principi appresi (Chiarezza, Specificità, Contesto, Formato, Esempi) sono la base su cui si costruiscono tutte le competenze avanzate dei capitoli successivi.

Punti Chiave da Ricordare:

- Prompt engineering è skill fondamentale, non opzionale
- I 5 principi sono checklist mentale, non formula rigida
- Iterazione è normale e desiderabile
- Template riutilizzabili massimizzano efficienza
- Queste skills abilitano tutto ciò che segue nel corso

Preparazione per Capitolo 4:

Nel prossimo capitolo, vedrete come scalare questi principi per gestire progetti completi attraverso lo Spec-Driven Development. Le competenze di prompt engineering che avete acquisito oggi diventeranno la foundation per scrivere specifiche funzionali che guidano implementazioni di qualità professionale.

Pratica Raccomandata:

- Applicate i 5 principi a 3-5 vostri task reali questa settimana
- Documentate prompt che funzionano bene per riutilizzo
- Osservate quanto tempo risparmiate con prompt ben strutturati
- Preparate domande su casi edge per Capitolo 4